

```

import tensorflow_datasets as tfds
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Load MNIST digits dataset (images of 0–9)
ds = tfds.load('mnist', split='train', as_supervised=True)

images = []
labels = []

# Convert dataset to numpy
for img, label in tfds.as_numpy(ds):
    images.append(img)
    labels.append(label)

images = np.array(images)
labels = np.array(labels)

# Flatten images and normalize
X = images.reshape(images.shape[0], -1) / 255.0 # (N, 784)
y = labels # Already 0–9

# One-hot encode labels
def one_hot_encode(y, num_classes=10):
    return np.eye(num_classes)[y]

Y = one_hot_encode(y)

# Add bias column
X_bias = np.hstack([np.ones((X.shape[0], 1)), X]) # (N, 785)

```

```
# Normal equation solution for beta
beta = np.linalg.pinv(X_bias.T @ X_bias) @ X_bias.T @ Y # (785, 10)

# Predictions
Y_pred = X_bias @ beta # (N, 10)
predicted_labels = np.argmax(Y_pred, axis=1)

# Training accuracy
accuracy = np.mean(predicted_labels == y) * 100
print(f"Training Accuracy: {accuracy:.2f}%")

# Show an example image and prediction
idx = 10
img = Image.fromarray(images[idx].squeeze(), mode='L')
img.show()
print(f"Label: {labels[idx]} -> Predicted: {predicted_labels[idx]}")
```